2021 SEMI Project

SQUARESPACE.COM/LOGO,
ICONS BY THE NOUN PROJECT

babmokja

Team name. 지금 배고프죠?

CONTENTS

01 프로젝트 소개

- 기획을 하게된 이유
- 프로젝트 간단 설명

02 모델

- 논리모델
- 물리모델
- 유스케이스다이어그램

03 파급효과

- 경제적효과
- 사회 · 문화적효과

04 향후전망

- 보완점
- 향후전망

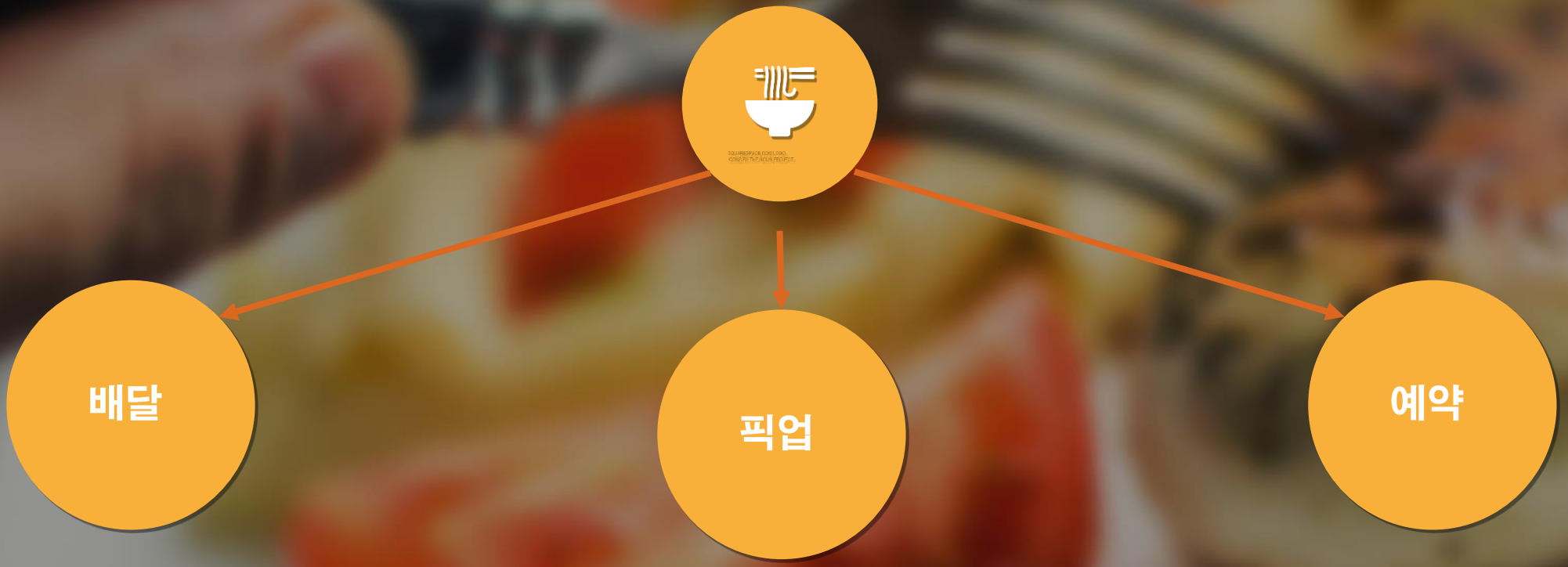
SQUARESPACE.COM/LOGO,
ICONS BY THE NOUN PROJECT

001

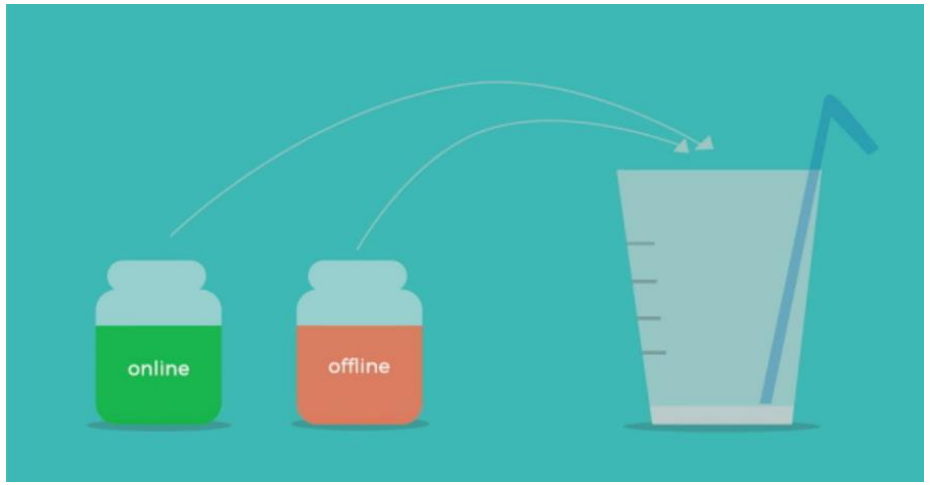
프로젝트 소개

babmokja

강남권 식당을 대상으로 배달, 픽업, 예약 사이트



▪ 기획을 하게된 이유



저희는 **강남권 식당을 대상으로** 배달, 픽업, 그리고 고객이 음식점에서 식사하고 싶은 **시간대**를 골라 **예약**을 해주는 기능을 도입하였습니다.

이런 기능을 통하여 저희 웹 사이트를 사용하는 고객들이 **맛집** 또한 간편하게 **배달, 픽업** 그리고 **예약**을 통해 이용할 수 있게 끔 도와주는 사이트입니다.

현재 코로나-19가 한국에도 큰 영향을 주고 있는 가운데 오프라인 업종들은 위기에 빠지게 되었습니다. 코로나로 인한 불안감 조성 등을 인해 근거를 들어도 과언이 아닙니다. 그리하여 생각한 방법이 **배달, 픽업** 그리고 **오프라인 예약 시스템** 모두 도입을 해보는게 어떨까라고 생각하였습니다.

현재 국내 배달 **O2O(Online to Offline)** 시장이 진화하고 있습니다.

O2O란 온라인과 오프라인을 **연계**하는 서비스를 말합니다.

온라인인 배달을 주로 시키는 치킨, 피자, 등과 함께 제한적인 배달, 픽업이 아닌 맛집 또한 가능할 수 있습니다.

오프라인인 배달 및 픽업 보다 가서 식사하고 싶은 고객층들에게는 **예약을 통해 웨이팅 시간 절약**을 통해 **온,오프라인**을 **동시에** 가능할 수 있게끔 기획하였습니다.

▪ 프로젝트 간단 설명

bobmokja
접속

회원가입
및
로그인

카테고리
검색
(시각화)

음식점
선택

예약
or
배달/픽업 선택

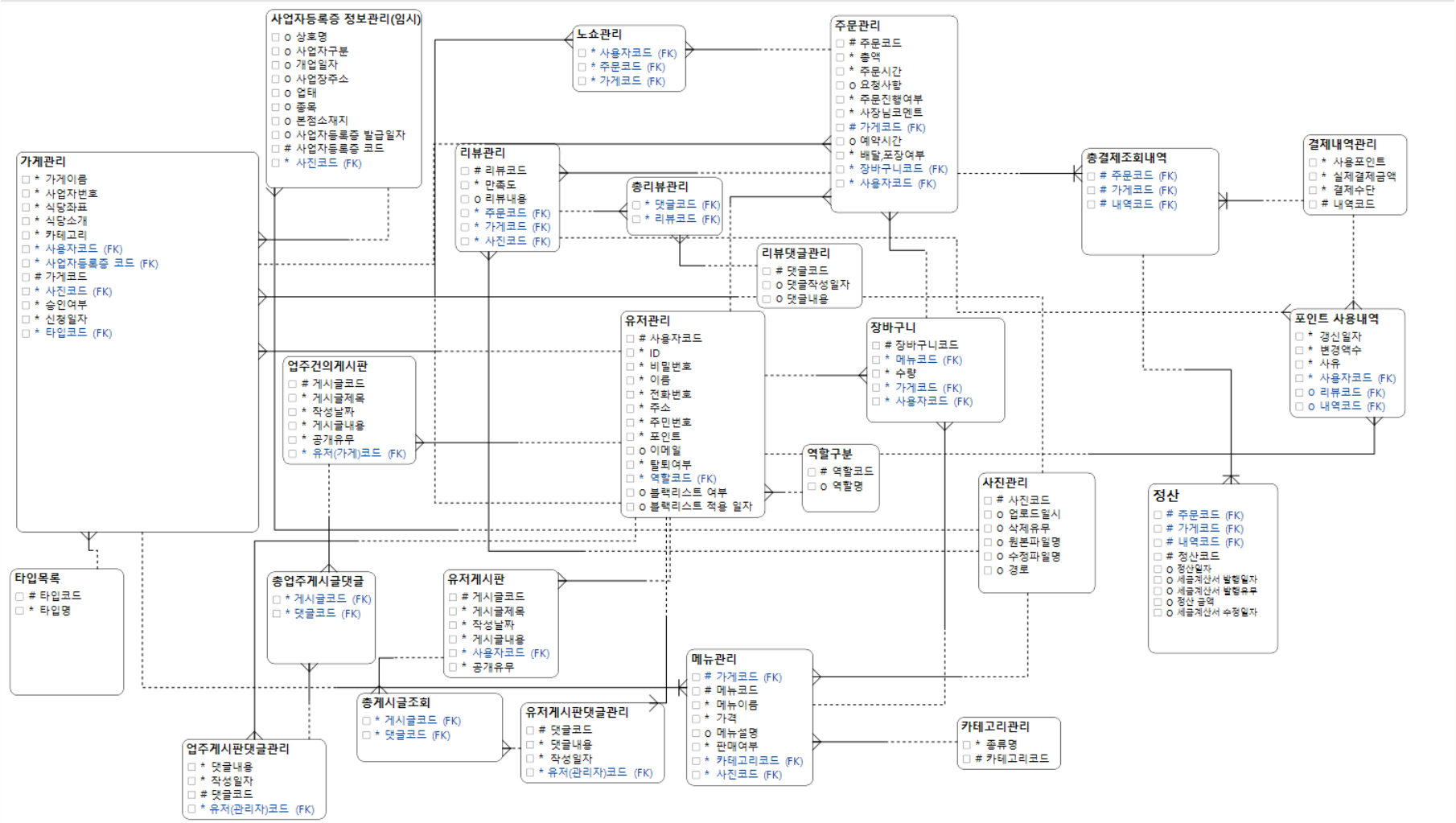
결제수단
선택
(카카오페이 **Or**
현금)

SQUARESPACE.COM/LOGO,
ICONS BY THE NOUN PROJECT

002

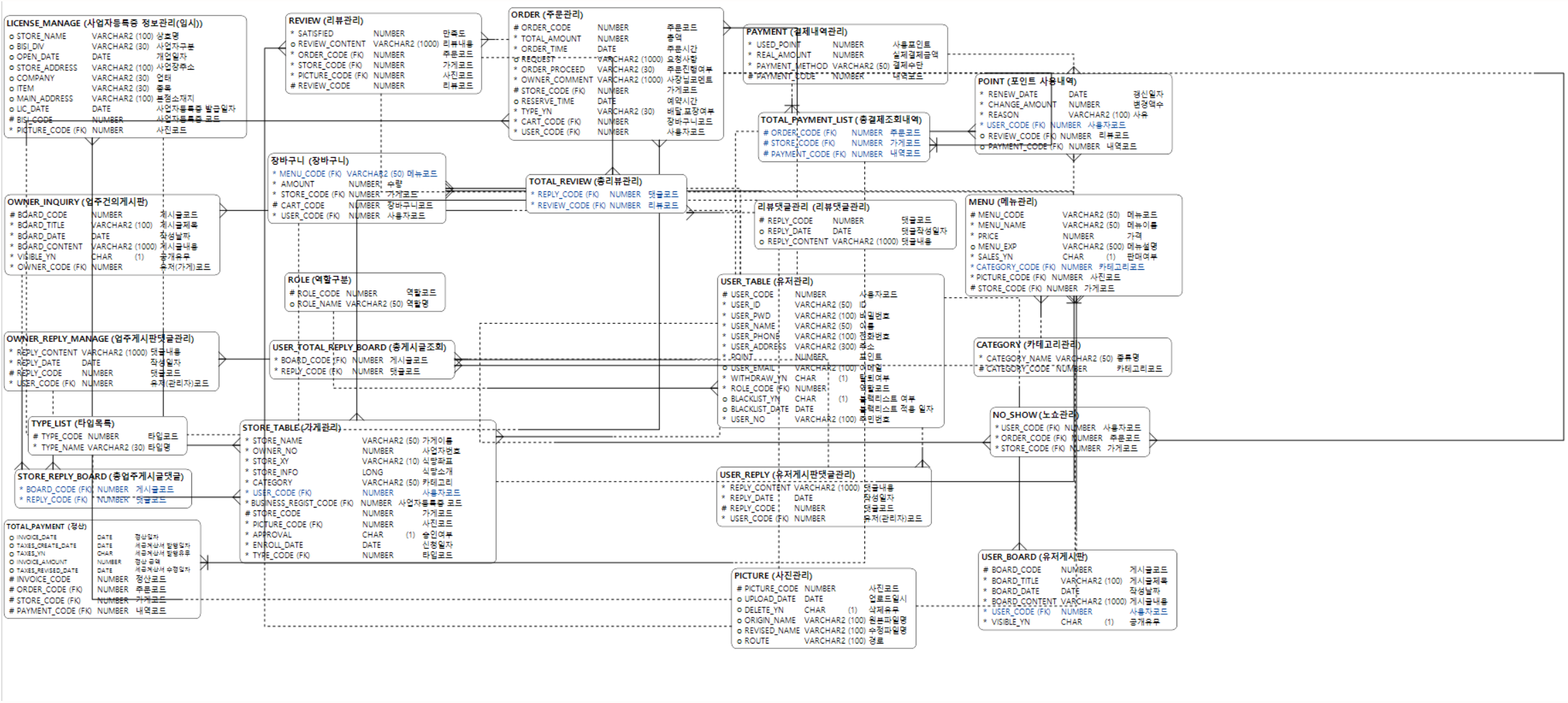
모델

▪ 논리 모델

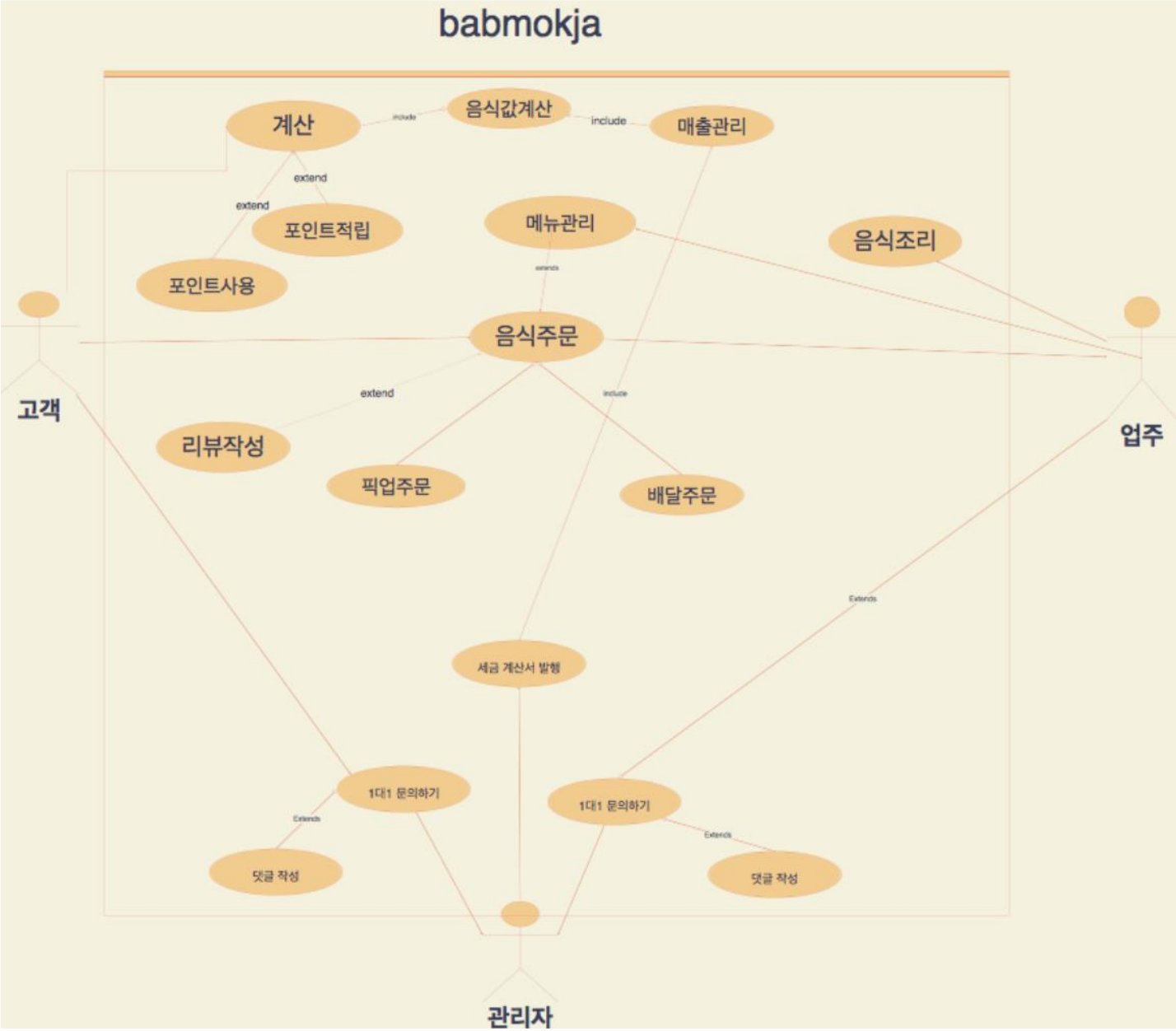

사업자등록증 정보관리(임시)
o 상호명
o 사업자구분
o 개업일자
o 사업장주소
o 업태
o 종목
o 본점소재지
o 사업자등록증 발급일자
사업자등록증 코드
* 사진코드 (FK)
노쇼관리
* 사용자코드 (FK)
* 주문코드 (FK)
* 가게코드 (FK)
주문관리
주문코드
* 총액
* 주문시간
o 요청사항
* 주문진행여부
* 사장님코멘트
# 가게코드 (FK)
o 예약시간
* 배달포장여부
* 장바구니코드 (FK)
* 사용자코드 (FK)
가게관리
* 가게이름
* 사업자번호
* 식당좌표
* 식당소개
* 카테고리
* 사용자코드 (FK)
* 사업자등록증 코드 (FK)
# 가게코드
* 사진코드 (FK)
* 승인여부
* 신청일자
* 타입코드 (FK)
리뷰관리
리뷰코드
* 만족도
o 리뷰내용
* 주문코드 (FK)
* 가게코드 (FK)
* 사진코드 (FK)
총리뷰관리
* 댓글코드 (FK)
* 리뷰코드 (FK)
총결제조회내역
주문코드 (FK)
가게코드 (FK)
내역코드 (FK)
결제내역관리
* 사용포인트
* 실제결제금액
* 결제수단
# 내역코드
리뷰댓글관리
댓글코드
o 댓글작성일자
o 댓글내용
유저관리
사용자코드
* ID
* 비밀번호
* 이름
* 전화번호
* 주소
* 주민번호
* 포인트
o 이메일
* 탈퇴여부
* 역할코드 (FK)
o 블랙리스트 여부
o 블랙리스트 적용 일자
장바구니
장바구니코드
* 메뉴코드 (FK)
* 수량
* 가게코드 (FK)
* 사용자코드 (FK)
포인트 사용내역
* 갱신일자
* 변경액수
* 사유
* 사용자코드 (FK)
o 리뷰코드 (FK)
o 내역코드 (FK)
업주건의게시판
게시글코드
* 게시글제목
* 작성날짜
* 게시글내용
* 공개유무
* 유저(가게)코드 (FK)
역할구분
역할코드
o 역할명
사진관리
사진코드
o 업로드일시
o 삭제유무
o 원본파일명
o 수정파일명
o 경로
정산
주문코드 (FK)
가게코드 (FK)
내역코드 (FK)
정산코드
o 정산일자
o 세금계산서 발행일자
o 세금계산서 발행유무
o 정산 금액
o 세금계산서 수정일자
타입목록
타입코드
* 타입명
총업주게시글댓글
* 게시글코드 (FK)
* 댓글코드 (FK)
유저게시판
게시글코드
* 게시글제목
* 작성날짜
* 게시글내용
* 사용자코드 (FK)
* 공개유무
메뉴관리
가게코드 (FK)
메뉴코드
* 메뉴이름
* 가격
o 메뉴설명
* 판매여부
* 카테고리코드 (FK)
* 사진코드 (FK)
총게시글조회
* 게시글코드 (FK)
* 댓글코드 (FK)
유저게시판댓글관리
댓글코드
* 댓글내용
* 작성일자
* 유저(관리자)코드 (FK)
카테고리관리
* 종류명
# 카테고리코드
업주게시판댓글관리
* 댓글내용
* 작성일자
# 댓글코드
* 유저(관리자)코드 (FK)

02 모델

▪ 물리 모델


LICENSE_MANAGE (사업자등록증 정보관리(임시))
o STORE_NAME VARCHAR2 (100) 상호명
o BISI_DIV VARCHAR2 (30) 사업자구분
o OPEN_DATE DATE 개업일자
o STORE_ADDRESS VARCHAR2 (100) 사업장주소
o COMPANY VARCHAR2 (30) 업태
o ITEM VARCHAR2 (30) 종목
o MAIN_ADDRESS VARCHAR2 (100) 본점소재지
o LIC_DATE DATE 사업자등록증 발급일자
BISI_CODE NUMBER 사업자등록증 코드
* PICTURE_CODE (FK) NUMBER 사진코드
REVIEW (리뷰관리)
* SATISFIED NUMBER 만족도
o REVIEW_CONTENT VARCHAR2 (1000) 리뷰내용
* ORDER_CODE (FK) NUMBER 주문코드
* STORE_CODE (FK) NUMBER 가게코드
* PICTURE_CODE (FK) NUMBER 사진코드
# REVIEW_CODE NUMBER 리뷰코드
ORDER (주문관리)
ORDER_CODE NUMBER 주문코드
* TOTAL_AMOUNT NUMBER 총액
* ORDER_TIME DATE 주문시간
o REQUEST VARCHAR2 (1000) 요청사항
* ORDER_PROCEED VARCHAR2 (30) 주문진행여부
* OWNER_COMMENT VARCHAR2 (1000) 사장님코멘트
# STORE_CODE (FK) NUMBER 가게코드
o RESERVE_TIME DATE 예약시간
* TYPE_YN VARCHAR2 (30) 배달,포장여부
* CART_CODE (FK) NUMBER 장바구니코드
* USER_CODE (FK) NUMBER 사용자코드
PAYMENT (결제내역관리)
* USED_POINT NUMBER 사용포인트
* REAL_AMOUNT NUMBER 실제결제금액
* PAYMENT_METHOD VARCHAR2 (50) 결제수단
# PAYMENT_CODE NUMBER 내역코드
POINT (포인트 사용내역)
* RENEW_DATE DATE 갱신일자
* CHANGE_AMOUNT NUMBER 변경액수
* REASON VARCHAR2 (100) 사유
* USER_CODE (FK) NUMBER 사용자코드
o REVIEW_CODE (FK) NUMBER 리뷰코드
o PAYMENT_CODE (FK) NUMBER 내역코드
TOTAL_PAYMENT_LIST (총결제조회내역)
ORDER_CODE (FK) NUMBER 주문코드
STORE_CODE (FK) NUMBER 가게코드
PAYMENT_CODE (FK) NUMBER 내역코드
장바구니 (장바구니)
* MENU_CODE (FK) VARCHAR2 (50) 메뉴코드
* AMOUNT NUMBER 수량
* STORE_CODE (FK) NUMBER 가게코드
# CART_CODE NUMBER 장바구니코드
* USER_CODE (FK) NUMBER 사용자코드
TOTAL_REVIEW (총리뷰관리)
* REPLY_CODE (FK) NUMBER 댓글코드
* REVIEW_CODE (FK) NUMBER 리뷰코드
OWNER_INQUIRY (업주건의게시판)
BOARD_CODE NUMBER 게시글코드
* BOARD_TITLE VARCHAR2 (100) 게시글제목
* BOARD_DATE DATE 작성날짜
* BOARD_CONTENT VARCHAR2 (1000) 게시글내용
* VISIBLE_YN CHAR (1) 공개유무
* OWNER_CODE (FK) NUMBER 유저(가게)코드
리뷰댓글관리 (리뷰댓글관리)
REPLY_CODE NUMBER 댓글코드
o REPLY_DATE DATE 댓글작성일자
o REPLY_CONTENT VARCHAR2 (1000) 댓글내용
MENU (메뉴관리)
MENU_CODE VARCHAR2 (50) 메뉴코드
* MENU_NAME VARCHAR2 (50) 메뉴이름
* PRICE NUMBER 가격
o MENU_EXP VARCHAR2 (500) 메뉴설명
* SALES_YN CHAR (1) 판매여부
* CATEGORY_CODE (FK) NUMBER 카테고리코드
* PICTURE_CODE (FK) NUMBER 사진코드
# STORE_CODE (FK) NUMBER 가게코드
ROLE (역할구분)
ROLE_CODE NUMBER 역할코드
o ROLE_NAME VARCHAR2 (50) 역할명
USER_TABLE (유저관리)
USER_CODE NUMBER 사용자코드
* USER_ID VARCHAR2 (50) ID
* USER_PWD VARCHAR2 (100) 비밀번호
* USER_NAME VARCHAR2 (50) 이름
* USER_PHONE VARCHAR2 (100) 전화번호
* USER_ADDRESS VARCHAR2 (300) 주소
* POINT NUMBER 포인트
o USER_EMAIL VARCHAR2 (100) 이메일
* WITHDRAW_YN CHAR (1) 탈퇴여부
* ROLE_CODE (FK) NUMBER 역할코드
o BLACKLIST_YN CHAR (1) 블랙리스트 여부
o BLACKLIST_DATE DATE 블랙리스트 적용 일자
* USER_NO VARCHAR2 (100) 주민번호
OWNER_REPLY_MANAGE (업주게시판댓글관리)
* REPLY_CONTENT VARCHAR2 (1000) 댓글내용
* REPLY_DATE DATE 작성일자
# REPLY_CODE NUMBER 댓글코드
* USER_CODE (FK) NUMBER 유저(관리자)코드
USER_TOTAL_REPLY_BOARD (총게시글조회)
* BOARD_CODE (FK) NUMBER 게시글코드
* REPLY_CODE (FK) NUMBER 댓글코드
CATEGORY (카테고리관리)
* CATEGORY_NAME VARCHAR2 (50) 종류명
# CATEGORY_CODE NUMBER 카테고리코드
TYPE_LIST (타입목록)
TYPE_CODE NUMBER 타입코드
* TYPE_NAME VARCHAR2 (30) 타입명
STORE_TABLE (가게관리)
* STORE_NAME VARCHAR2 (50) 가게이름
* OWNER_NO NUMBER 사업자번호
* STORE_XY VARCHAR2 (10) 식당좌표
* STORE_INFO LONG 식당소개
* CATEGORY VARCHAR2 (50) 카테고리
* USER_CODE (FK) NUMBER 사용자코드
* BUSINESS_REGIST_CODE (FK) NUMBER 사업자등록증 코드
# STORE_CODE NUMBER 가게코드
* PICTURE_CODE (FK) NUMBER 사진코드
* APPROVAL CHAR (1) 승인여부
* ENROLL_DATE DATE 신청일자
* TYPE_CODE (FK) NUMBER 타입코드
NO_SHOW (노쇼관리)
* USER_CODE (FK) NUMBER 사용자코드
* ORDER_CODE (FK) NUMBER 주문코드
* STORE_CODE (FK) NUMBER 가게코드
STORE_REPLY_BOARD (총업주게시글댓글)
* BOARD_CODE (FK) NUMBER 게시글코드
* REPLY_CODE (FK) NUMBER 댓글코드
USER_REPLY (유저게시판댓글관리)
* REPLY_CONTENT VARCHAR2 (1000) 댓글내용
* REPLY_DATE DATE 작성일자
# REPLY_CODE NUMBER 댓글코드
* USER_CODE (FK) NUMBER 유저(관리자)코드
TOTAL_PAYMENT (정산)
o INVOICE_DATE DATE 정산일자
o TAXES_CREATE_DATE DATE 세금계산서 발행일자
o TAXES_YN CHAR 세금계산서 발행유무
o INVOICE_AMOUNT NUMBER 정산금액
o TAXES_REVISED_DATE DATE 세금계산서 수정일자
INVOICE_CODE NUMBER 정산코드
ORDER_CODE (FK) NUMBER 주문코드
STORE_CODE (FK) NUMBER 가게코드
PAYMENT_CODE (FK) NUMBER 내역코드
PICTURE (사진관리)
PICTURE_CODE NUMBER 사진코드
o UPLOAD_DATE DATE 업로드일시
o DELETE_YN CHAR (1) 삭제유무
o ORIGIN_NAME VARCHAR2 (100) 원본파일명
o REVISED_NAME VARCHAR2 (100) 수정파일명
o ROUTE VARCHAR2 (100) 경로
USER_BOARD (유저게시판)
BOARD_CODE NUMBER 게시글코드
* BOARD_TITLE VARCHAR2 (100) 게시글제목
* BOARD_DATE DATE 작성날짜
* BOARD_CONTENT VARCHAR2 (1000) 게시글내용
* USER_CODE (FK) NUMBER 사용자코드
* VISIBLE_YN CHAR (1) 공개유무

▪ 유스케이스 다이어그램


babmokja
계산
음식값계산
매출관리
포인트적립
메뉴관리
음식조리
포인트사용
음식주문
고객
업주
리뷰작성
픽업주문
배달주문
세금 계산서 발행
1대1 문의하기
1대1 문의하기
댓글 작성
댓글 작성
관리자

SEVICE

황준호

```java
package com.BoB.mvc.owner.model.service;

import static com.BoB.mvc.common.jdbc.JDBCTemplate.close;

public class OwnerService {

    private final OwnerDAO ownerDAO;
    private orderDAO orderDAO;

    public OwnerService() {
        orderDAO = new orderDAO();
        ownerDAO = new OwnerDAO();
    }



    public int registOwner(Map<String, PictureDTO> picture, LicenseMan

        Connection con = getConnection();
        int result5 = 0;

        int result1 = ownerDAO.insertPicture(con, picture);
        System.out.println("result1 : "+ result1);
        if(result1==2) {commit(con);}
        else {rollback(con);}

        int result2 = ownerDAO.insertLicenseManager(con,lm,picture);
        System.out.println("result2 : " + result2);
        if(result1==2 && result2>0) {commit(con);}
        else {rollback(con);}

        int result3 = ownerDAO.insertOwner(con, owner);
        System.out.println("result3"+result3);
        if(result1==2 && result2>0&&result3>0) {commit(con);}
        else {rollback(con);}

        int result4 = ownerDAO.insertStore(con,store,picture,owner,lm);
        System.out.println("result4"+result4);
        if(result1==2 && result2>0&&result3>0&&result4>0) {commit(con);}
        else {rollback(con);}

        result5 = ownerDAO.insertPICtable(con, store, picture, owner, lm);
        System.out.println("result5"+result5);
        if(result1==2 && result2>0&&result3>0&&result4>0&&result5>0) {commit(con);}
        else {rollback(con);}

        close(con);

        return result5;
    }
```

```java
package com.BoB.mvc.owner.model.service;

import static com.BoB.mvc.common.jdbc.JDBCTemplate.close;

public class MenuListService {

    private final MenuListDAO menuListDAO;

    public MenuListService() {
        menuListDAO = new MenuListDAO();
    }

    public List<MenuListDTO> selectAllMenuList(StoreInfoDTO storeInfoDTO) {

        Connection con = getConnection();

        List<MenuListDTO> menuList = menuListDAO.selectAllMenuList(con,storeInfoDTO);

        close(con);

        return menuList;
    }

    public int insertBoard(MenuListDTO newMenu, PictureDTO pictureDTO, StoreInfoDTO

        Connection con = getConnection();
        int result = 0;
        int pictureResult = menuListDAO.insertPicture(con,pictureDTO);

        int pictureNo = menuListDAO.selectPictureSequence(con);

        newMenu.setPicCode(pictureNo);

        int newMenuResult = menuListDAO.insertNewMenu(con,newMenu,storeInfoDTO);

        System.out.println("#######################picture result : " + pictureRes
        System.out.println("#######################newMenu result : " + newMenuRes


        if(pictureResult > 0 && newMenuResult >0) {
            commit(con);
            result = 1;
            System.out.println("JAMES JJANG JJANG");
        }else {
            rollback(con);
        }

        return result;
    }

    public String getMenuname(MenuListDTO menuList) {

        Connection con = getConnection();
```

```java
package com.BoB.mvc.owner.model.service;

import static com.BoB.mvc.common.jdbc.JDBCTemplate.close;

public class SuggestionService {

    private final SuggestionDAO suggestionDAO;

    public SuggestionService(){
        suggestionDAO = new SuggestionDAO();
    }
    public List<SuggestionDTO> selectAllSuggestion(PageInfoDTO pageInfo, int ownerCode) {

        Connection con = getConnection();

        List<SuggestionDTO> suggestionList = suggestionDAO.selectAllList(con, pageInfo,ownerCode);

        close(con);


        return suggestionList;
    }
    public int insertSuggestion(SuggestionDTO suggestionDTO) {

        Connection con =getConnection();

        int result = suggestionDAO.insertSuggestion(con,suggestionDTO);

        if(result>0) {
            commit(con);
        }else {
            rollback(con);
        }
        close(con);

        return result;
    }
    public SuggestionDTO selectDetailSuggestion(String boardNo) {

        Connection con = getConnection();

        SuggestionDTO suggestionDetail = suggestionDAO.selectDetail(con,boardNo);

        close(con);
        return suggestionDetail;
    }

    public int selectSuggestionTotalCount(int ownerCode) {
```

MAPPER

황준호

```
    SELECT
           SEQ_PICTURE_CODE.CURRVAL
      FROM DUAL
</entry>

<entry key="selectDailySales">
    SELECT A.RNUM
         , A.ORDER_CODE
         , A.ORDER_TIME
         , A.TOTAL_AMOUNT
         , A.STORE_CODE
         , A.TYPE_YN
      FROM (SELECT ROWNUM.RNUM
                 , B.ORDER_CODE
                 , B.ORDER_TIME
                 , B.TOTAL_AMOUNT
                 , B.STORE_CODE
                 , B.TYPE_YN
              FROM (SELECT O.ORDER_CODE
                         , O.ORDER_TIME
                         , O.TOTAL_AMOUNT
                         , O.STORE_CODE
                         , O.TYPE_YN
                     FROM ORDER_TABLE O
                    WHERE EXTRACT(DAY FROM (ORDER_TIME))=?
                      AND STORE_CODE = ?
                    ORDER BY ORDER_TIME) B
                    ) A
     WHERE A.RNUM BETWEEN ? AND ?
</entry>
<entry key="selectAllSales">
    SELECT *
      FROM ORDER_TABLE
     ORDER BY ORDER_TIME

</entry>

<entry key="selectMonthlySales">
    SELECT
           EXTRACT(MONTH FROM (ORDER_TIME)) as "MONTH"
         , count(ORDER_CODE) as "COUNT"
         , sum(TOTAL_AMOUNT) as "TOTAL"
      FROM ORDER_TABLE
     GROUP BY EXTRACT(MONTH FROM (ORDER_TIME))
     ORDER BY EXTRACT(MONTH FROM (ORDER_TIME))

</entry>

<entry key = "selectAllSuggestion">
    SELECT A.RNUM
```

DAO

황준호

```java
package com.BoB.mvc.owner.model.dao;

import static com.BoB.mvc.common.jdbc.JDBCTemplate.close

public class OwnerDAO {

    private Properties prop = new Properties();

    public OwnerDAO() {

        try {
            prop.loadFromXML(new FileInputStream(ConfigL
        } catch (IOException e) {
            e.printStackTrace();
        }

    }



    public int insertPicture(Connection con, Map<String,

        PreparedStatement pstmt1 = null;
        PreparedStatement pstmt2 = null;
        int result1 = 0;
        int result2 = 0;

        String query1 = prop.getProperty("registPictures


        try {
            pstmt1 =con.prepareStatement(query1);

            pstmt1.setString(1, picture.get("pict1").get
            pstmt1.setString(2, picture.get("pict1").get
            pstmt1.setString(3, picture.get("pict1").get
            result1 =pstmt1.executeUpdate();

            pstmt2 =con.prepareStatement(query1);

            pstmt2.setString(1, picture.get("pict2").getOriginName());
            pstmt2.setString(2, picture.get("pict2").getRevisedName());
            pstmt2.setString(3, picture.get("pict2").getRoute());

            result2 = pstmt2.executeUpdate();


        } catch (SQLException e) {
            e.printStackTrace();
        } finally {
            close(pstmt1);
            close(pstmt2);
        }
```

```java
package com.BoB.mvc.owner.model.dao;

import static com.BoB.mvc.common.jdbc.JDBCTemplate.close;

public class MenuListDAO {

    private final Properties prop;

    public MenuListDAO() {
        prop = new Properties();

        try {
            prop.loadFromXML(new FileInputStream(ConfigLocation.MAPPER_LOCATION+"owner/owner-mapper.xml"));
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    public List<MenuListDTO> selectAllMenuList(Conn

        PreparedStatement pstmt = null;
        ResultSet rset = null;

        List<MenuListDTO> menuList = null;

        String query = prop.getProperty("selectAllMe

        try {
            pstmt = con.prepareStatement(query);
            pstmt.setInt(1, storeInfoDTO.getStoreCode
            rset = pstmt.executeQuery();

            menuList = new ArrayList<>();

            while(rset.next()) {

                MenuListDTO menu = new MenuListDTO();
                menu.setPictureCode(new PictureDTO());
//                menu.setStoreCode(new Stor);


                menu.setMenuCode(rset.getString("MENU_
                menu.setMenuName(rset.getString("MENU_
                menu.setPrice(rset.getInt("PRICE"));
                menu.setMenuExplain(rset.getString("ME
                menu.setSalesYN(rset.getString("SALES_
                menu.setStoCode(rset.getInt("STORE_COD
//                menu.getStoreCode().setStoreNum(rset

                menu.getPictureCode().setOriginName(rs
                menu.getPictureCode().setRoute(rset.ge
```

```java
package com.BoB.mvc.owner.model.dao;

import static com.BoB.mvc.common.jdbc.JDBCTemplate.close;

public class SuggestionDAO {

    private final Properties prop;

    public SuggestionDAO(){
        prop = new Properties();

        try {
            prop.loadFromXML(new FileInputStream(ConfigLocation.MAPPER_LOCATION+"owner/owner-mapper.xml"));
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    public List<SuggestionDTO> selectAllList(Connection con, PageInfoDTO pageInfo, int ownerCode) {

        PreparedStatement pstmt = null;
        ResultSet rset = null;

        List<SuggestionDTO> suggestionList = null;

        String query = prop.getProperty("selectAllSuggestion");

        try {
            suggestionList = new ArrayList<>();
            pstmt= con.prepareStatement(query);
            pstmt.setInt(1, ownerCode);
            pstmt.setInt(2, pageInfo.getStartRow());
            pstmt.setInt(3, pageInfo.getEndRow());

            rset = pstmt.executeQuery();

            while(rset.next()) {
                SuggestionDTO suggestion = new SuggestionDTO();

                suggestion.setBoardCode(rset.getInt("BOARD_CODE"));
                suggestion.setBoardTitle(rset.getString("BOARD_TITLE"));
                suggestion.setBoardDate(rset.getDate("BOARD_DATE"));

                suggestionList.add(suggestion);
            }
        } catch (SQLException e) {
            e.printStackTrace();
        } finally {
            close(rset);
            close(pstmt);
        }
```

황준호

ENCRYPT
래퍼처리

```java
import java.util.Map;

public class EncryptReqeustWrapper extends HttpServletRequestWrapper{


    public EncryptReqeustWrapper(HttpServletRequest request) {
        super(request);
    }

    @Override
    public String getParameter(String key) {

        String value = "";

        if("userPwd".equals(key)) {

            BCryptPasswordEncoder passwordEncoder = new BCryptPasswordEncoder();
            value = passwordEncoder.encode(super.getParameter(key));
        } else if("pw".equals(key)) {

            BCryptPasswordEncoder passwordEncoder = new BCryptPasswordEncoder();
            value = passwordEncoder.encode(super.getParameter(key));
            System.out.println("Encrypt처리 완료");
        } else {
            value = super.getParameter(key);
        }
        return value;
    }


}
```

getParameter 오버라이드로
“userPwd”와 “pw”를 key로 가질 때
getParameter시 Encrypt 래퍼처리

회원가입
비동기 처리 – 아이디 중복 확인

황준호

```java
package com.BoB.mvc.owner.controller;

import java.io.IOException;

/**
 * Servlet implementation class IdCheckServlet
 */
@WebServlet("/owner/idCheck")
public class IdCheckServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {

        String memberId = request.getParameter("memberId");

        System.out.println("memberId : " + memberId);

        OwnerService Service = new OwnerService();

        String checkId = Service.IdCheck(memberId);
        response.setCharacterEncoding("UTF-8");
        PrintWriter out = response.getWriter();

        System.out.println("memberId :" + memberId);
        System.out.println("checkId : " + checkId);

        if(memberId.equals(checkId)) {

            System.out.println("request.getContextPath() : " + request.getServletPath());
            String a = "1";
            out.print(a);

        } else {
            out.print("2");
        }

        out.flush();
        out.close();
    }

}
```

```html
body style="overflow:scroll;">

div class="mainsec">
        <label class="bigbtn" >밥묵자 사장님 회원가입</label>
        <center>
      <div id='signin'>
   <form id="joinForm" action="${ pageContext.servletContext.contextPath }/member/owner/regist" method="post" encType="multipart/form-data"
        <table cellpadding="10" cellspacing="15" style="margin-left: 180px;" >
           <tr >
             <td>아이디 </td>
             <td><input type="text" size = "25"style="padding:4px;font-size:13px;border-radius: 10px;" name="memberId" id="memberId" onchang
                 <input type="button" id="overlap" style="display:none;" >
                 <label for="overlap" id="idCheck">중복확인</label>
                 <script>
                  $("#idCheck").click(function(){

                      const memberId = document.getElementById("memberId").value;


                      $.ajax({
                         url: "${ pageContext.servletContext.contextPath }/owner/idCheck",
                         type: "get",
                         data : { memberId: memberId },
                         success: function(data, textStatus, xhr) {
                            console.table(data);
                            if(data == "1"){
                            alert("중복된 아이디입니다.");
                            }
                            else {
                                alert("사용가능한 아이디입니다.");
                            }
                         },
                         error: function(xhr, status, error) {
                            console.log(error);
                         }
                      });
                  });


             </script>
             </td>

            <td>비밀번호 </td>
            <td>><input type="password" id="pw" onchange="mempwd();" name="pw"  class="chk" size="25" placeholder="영문자와 숫자만 사용 가능" style=
           </tr>
```

회원가입 – MULTIPART FORM

황준호

```java
OwnerDTO owner = new OwnerDTO();
owner.setMemberId(parameter.get("memberId"));
String value = "";
BCryptPasswordEncoder passwordEncoder = new BCryptPasswordEncoder();

value = passwordEncoder.encode(parameter.get("pw"));
owner.setMemberPwd(value);
owner.setMemberPwd(parameter.get("pw"));
```

```java
/*
 * 최종적으로 request를 parsing하고 파일을 저장한 뒤 필요한 내용을 담을 리스트와 맵이다.
 * 파일에 대한 정보는 리스트(fileList)에 다른 파라미터의 정보는 모두 맵에 담을 것이다.
 */

/* 파일이 여러개일 수 있으니 리스트 / 파일에대한 정보 속성을 키밸류값으로 갖고있는것을 리스트에 담는다.*/
List<Map<String,String>> fileList = new ArrayList<>();
/* 파일결과값말고 파라미터 담아주는 공간 / parameter, fileList 이들을 db에 넣어준다.*/
Map<String,String> parameter = new HashMap<>();


-------------------------------------------------


/* 파일에 대한 속성정보를 commons는 객체단위로 만들어놔야함 */
/* 파일을 업로드할 때 최대 크기나 임시 저장할 폴더의 경로 등을 포함하기 위한 인스턴스 */
/* 즉, fileItemFactory는 commons의 속성정보를 담고있다. */
DiskFileItemFactory fileItemFactory = new DiskFileItemFactory();
/* 저장공간 */
fileItemFactory.setRepository(new File(fileUploadDirectory));
fileItemFactory.setSizeThreshold(maxFileSize);
```

```java
if(!item.isFormField()) { // 파일 데이터인 경우

    -------------------------- 업로드 파일 정제작업 (파일명 필터링, 확장자 필터링) ------------------
    if(item.getSize() > 0) {
        /*
         * 파일의 사이즈가 0보다 커야 전송된 파일이 있다는 의미
         * 전송된 파일이 있는 경우에만 처리하고, 0인 경우에는 무시하도록 로직을 작성하겠다.
         */
        String filedName = item.getFieldName(); // 태그 name값
        String originFileName = item.getName(); // 파일원본이름

        System.out.println("filedName : " + filedName);
        System.out.println("originFileName : " + originFileName);

        /*
         * 업로드되는 파일명을 변경작업
         * 1. 동일한 파일명이 들어왔을 때를 대비
         * 2. 특수문자 또는 입력받기 힘든 문자를 받았을 때를 대비 (업로드가 안되기때문)
         */

        int dot = originFileName.lastIndexOf(".");
        System.out.println("dot : " +  dot);
        String ext = originFileName.substring(dot); // substring 지정한 시작점부터 끝까지 가져온다
        System.out.println("ext : " + ext);
                                                    /* randomUUID로 인한 특수문자('-') 제거 */
        String randomFileName = UUID.randomUUID().toString().replace("-", "") + ext;

        /* 저장할 파일 정보를 담을 인스턴스를 생성하고 ***지금까지 중복방지처리한 파일명을 넣어 파일생성경로를 설정한다.**** */
        File storeFile = new File(fileUploadDirectory + "/" + randomFileName);

        // 삭제테스트를 위해서 강제로 에러를 발생 시킴
        String str = "";
        System.out.println(str.charAt(1));

        /* **************저장한다.********************** */
        item.write(storeFile);

        ------------------------여기까지 file upload 끝!!!!  이제 DB에 넘겨줄 값을 담아줄것이다. ------------------

        이제 DB에 넘겨줄 값을 담아줄것이다. DB에 넣어줄 샘플링 파일 업로드해줬을때 저장해줄 기본정보 , 파일업로드날짜 수정된날짜, 삭제여

        /* 필요한 정보를 Map에 담는다. */
        Map<String, String> fileMap = new HashMap<>();
        fileMap.put("fileName", filedName);             // 태그 name값
        fileMap.put("originFileName", originFileName);  // 파일 원본이름
        fileMap.put("saveFileName", randomFileName);    // 저장된 파일이름
        fileMap.put("savePath", fileUploadDirectory);   // 파일 저장경로

        fileList.add(fileMap);
```

멀티파트 폼 데이터인 경우 Post방식으로 보내서 getParameter로 읽는 구조가 적용되지 않았다.

따라서

1. 맵방식으로 데이터를 꺼내야했다.
2. Encrypt래퍼의 getParameter 오버라이드 적용 불가로 새로운 encrypt 객체 생성해서 적용하였다.

```java
    } else {
        // form 데이터인 경우
        /*
         * 전송된 폼의 name은 getFieldName()으로 받아오고,
         * 해당 필드의 value는 getString()으로 받아온다.
         * 보내는 곳에서 인코딩 설정을 하더라도 받는곳에서는 전송되는 파라미터를 ISO-8859-1로 처리한다.
         * "별도로" ISO-8859-1로 해석된 한글을 UTF-8로 변경해주어야 한다.
         */
        request.setCharacterEncoding("UTF-8");
        parameter.put(item.getFieldName(), item.getString());
        ISO8859로 받고 0101바이트로 바꾸고(.getBytes) 그 0101바이트를 new String으로 UTF-8로 바꾼다는 뜻
        parameter.put(item.getFieldName(), new String(item.getString().getBytes("ISO-8859-1"),"UTF-8")
        /* ******************** 무조건 이렇게 해줘야한다.********************** */
    }
}

System.out.println("parameter : " + parameter); // form 태그의 value값을 넣어줬다.
System.out.println("fileList : " + fileList);


---------- PICTURE테이블-------

---사업자등록증---
PictureDTO pict1 = new PictureDTO();
pict1.setOriginName(fileList.get(0).get("originFileName"));
pict1.setRevisedName(fileList.get(0).get("saveFileName"));
pict1.setRoute(fileList.get(0).get("savePath"));

System.out.println(pict1);

---가게사진---
PictureDTO pict2 = new PictureDTO();
pict2.setOriginName(fileList.get(1).get("originFileName"));
pict2.setRevisedName(fileList.get(1).get("saveFileName"));
pict2.setRoute(fileList.get(1).get("savePath"));

System.out.println(pict2);

Map<String, PictureDTO> picture = new HashMap<String, PictureDTO>();
picture.put("pict1", pict1);
picture.put("pict2", pict2);


---------LICENSE MANAGE 테이블-----

LicenseManagerDTO lm = new LicenseManagerDTO();
lm.setSbName(parameter.get("sbName"));
lm.setBISI(parameter.get("BISI"));
lm.setOpenDate(Date.valueOf(parameter.get("openDate")));
```

황준호

회원가입
Pk,Fk 관계에 따른 insert 순서

발생 순서를 고려하여 순서에 맞게 테이블 생성

```java
package com.BoB.mvc.owner.model.service;

import static com.BoB.mvc.common.jdbc.JDBCTemplate.close;

public class OwnerService {

    private final OwnerDAO ownerDAO;
    private orderDAO orderDAO;

    public OwnerService() {
        orderDAO = new orderDAO();
        ownerDAO = new OwnerDAO();
    }



    public int registOwner(Map<String, PictureDTO> picture, LicenseManagerDTO lm, OwnerDTO owner, StoreInfoDTO store) {

        Connection con = getConnection();
        int result5 = 0;

        int result1 = ownerDAO.insertPicture(con, picture);
        System.out.println("result1 : "+ result1);
        if(result1==2) {commit(con);}
        else {rollback(con);}

        int result2 = ownerDAO.insertLicenseManager(con,lm,picture);
        System.out.println("result2 : " + result2);
        if(result1==2 && result2>0) {commit(con);}
        else {rollback(con);}

        int result3 = ownerDAO.insertOwner(con, owner);
        System.out.println("result3"+result3);
        if(result1==2 && result2>0&&result3>0) {commit(con);}
        else {rollback(con);}

        int result4 = ownerDAO.insertStore(con,store,picture,owner,lm);
        System.out.println("result4"+result4);
        if(result1==2 && result2>0&&result3>0&&result4>0) {commit(con);}
        else {rollback(con);}

        result5 = ownerDAO.insertPICtable(con, store, picture, owner, lm);
        System.out.println("result5"+result5);
        if(result1==2 && result2>0&&result3>0&&result4>0&&result5>0) {commit(con);}
        else {rollback(con);}

        close(con);

        return result5;
    }
```

```xml
<entry key="registPictures">
INSERT
  INTO PICTURE
VALUES
(
  SEQ_PICTURE.NEXTVAL
, SYSDATE
, 'N'
, ?
, ?
, ?
)
</entry>

<entry key="registLicenseManager">
INSERT
  INTO LICENSE_MANAGE
VALUES
(
  ?
, ?
, ?
, ?
, ?
, ?
, ?
, ?
, SEQ_LMANAGER.NEXTVAL
, SEQ_PICTURE.CURRVAL-1
)
</entry>
<entry key="registOwner">
INSERT
  INTO USER_TABLE
  VALUES
  (
    SEQ_USER.NEXTVAL
  , ?
  , ?
  , ?
  , ?
  , ?
  , DEFAULT
  , ?
  , 'N'
  , '2'
  , 'N'
  , NULL
  , ?
  )
</entry>
<entry key="registStore">
```

회원가입 및 정보수정
비밀번호 중복 체크

황준호

```
function mempwd() {


    //****** .은 아무문자 들어와도 되는 위치 잡는 역할 .개수 그 이상도 가능하다. 이하는 불가능

    var regExp = /^[A-Z\da-z]{6,16}$/;
    var regExp2 = /^[0-9]{6,16}$/;

    var pno = document.getElementById("pw").value;



    if (regExp.test(pno)||regExp2.test(pno)) {
      alert("정상입력");
    } else {
      alert("6자리이상 16자리이하로 입력하세요.(특수문자 사용불가)");
    }
}
```

비밀번호 유효성 검증

```
function check_pw(){
    if(document.getElementById("pw").value === document.getElementById("pw2").value){
         document.getElementById("check").innerHTML="비밀번호가 일치";
         document.getElementById("check").style.color='blue';

     }
     else{
         document.getElementById("check").innerHTML="비밀번호 불일치.";
         document.getElementById("check").style.color="red";

     }
}
```

비밀번호 중복체크

Id, 이름, 주민번호 유효성 처리

황준호

```
function memid() {

  //****** .은 아무문자 들어와도 되는 위치 잡는 역할 .개수 그 이상도 가능하다. 이하는 불가능

  var regExp = /^[a-z^A-Z]{4}[a-z\dA-Z^]{0,}$/;

  var pno = document.getElementById("memberId").value;

  // 개수만 맞으면 true 반환
  console.log(regExp.test(pno));
  if (regExp.test(pno)) {
    document.getElementById("memberId").style.borderColor = "blue";
    document.getElementById("memberId").style.backgroundColor = "white";
  } else {
    alert("영문자 4자리로 시작해야합니다.(특수문자 사용불가)");
    document.getElementById("memberId").style.backgroundColor = "red";
  }
  }


function namecheck() {

  var regExp = /^[가-힣^]{3,}$/;

  var pno = document.getElementById("named").value;

  // 개수만 맞으면 true 반환
  console.log(regExp.test(pno));
  if (regExp.test(pno)) {
    alert("정상입력");
  } else {
    alert("한글로 입력해주세요");
  }
}

function resi() {


  //****** .은 아무문자 들어와도 되는 위치 잡는 역할 .개수 그 이상도 가능하다. 이하는 불가능

  var regExp = /\d\d\d\d\d\d-[1234]\d\d\d\d\d\d/;
  var regExp2 = /\d\d\d\d\d\d-[1234]\d\d\d\d\d\d/;

  var pno = document.getElementById("residentNum").value;

  // 개수만 맞으면 true 반환
  console.log(regExp.test(pno));
  if (regExp.test(pno)||regExp2.test(pno)) {
    alert("정상입력");
  } else {
    alert("잘못 입력");
```

3글자 이상

페이징 처리

황준호

페이징의 공통부분 클래스화

```java
package com.BoB.mvc.owner.paging;

import com.BoB.mvc.owner.model.dto.PageInfoDTO;

public class OwnerPaging {

    public static PageInfoDTO getPageInfo(int pageNo, int totalCount, int limit, int buttonAmount) {

        /* PageNo와 totalCount는 매개변수로 넘어온 상태이다.*/

        int maxPage;      // 전체 페이지에서 가장 마지막 페이지
        int startPage;    // 한번에 표시될 페이지 버튼의 시작할 페이지
        int endPage;      // 한번에 표시될 페이지 버튼의 끝나는 페이지
        int startRow;
        int endRow;

        /* 총 페이지 수 계산*/
        //maxPage = (int)((double)totalCount/ limit + 0.9);
        maxPage =  (int) Math.ceil((double)totalCount/limit);

        /* 현재 페이지에 보여줄 시작 페이지 수 */
        /* 예를 들어서 10개씩 보여줄려고한다. 1, 11, 21, 31*/
        //startPage = ((int)((double)pageNo / buttonAmount + 0.9) -1) * buttonAmount + 1;
        startPage = (int)(Math.ceil((double) pageNo / buttonAmount) - 1) * buttonAmount + 1;

        /* 목록 아래쪽에 보여질 마지막 페이지 수 */
        endPage = startPage + buttonAmount - 1;

        if(maxPage < endPage) {
            endPage = maxPage;
        }

        /* 마지막 페이지는 0이 될 수 없기 때문에 게시물이 아무것도 존재하지 않으면 max페이지와 endPage는 0이된다.*/
        if(maxPage == 0 && endPage == 0) {
            maxPage = startPage;
            endPage = startPage;
        }

        /* 조회할 시작 번호와 마지막 행 번호를 계산 */
        startRow = (pageNo - 1) * limit + 1; // 4페이지 (4-1) * 10  31 1 1
        endRow = startRow + limit -1;

        PageInfoDTO pageInfo = new PageInfoDTO(pageNo,totalCount,limit,buttonAmount,maxPage,startPage,endPage,startRow,endRow);

        return pageInfo;
    }
}
```

페이징 상세검색 처리

```java
public List<ReviewBoardListDTO> selectSearchReviewList(Connection con, String condition, String value,
        PageInfoDTO pageInfo, StoreInfoDTO storeDTO) {

    PreparedStatement pstmt = null;
    ResultSet rset = null;


    String query = null;
    List<ReviewBoardListDTO> reviewSearchList = null;

    if(condition.equals("satisfied")) {

        query = prop.getProperty("searchsatisfiedBoardCount");
    }else if(condition.equals("reviewContent")) {

        query = prop.getProperty("searchreviewContentBoardCount");
    }else if(condition.equals("userId")) {

        query = prop.getProperty("searchuserIdBoardCount");
    }

    if(value.equals("보통")) {
        value = "1" ;
    } else if(value.equals("좋음")) {
        value = "2" ;
    } else if(value.equals("매우좋음")) {
        value = "3" ;
    }


    try {
        pstmt = con.prepareStatement(query);
        pstmt.setInt(1, storeDTO.getStoreCode());
        pstmt.setString(2, value);
        pstmt.setInt(3, pageInfo.getStartRow());
        pstmt.setInt(4, pageInfo.getEndRow());

        rset = pstmt.executeQuery();

        reviewSearchList = new ArrayList<>();

        while(rset.next()) {
            ReviewBoardListDTO review = new ReviewBoardListDTO();
            review.setRowNum(rset.getInt("RNUM"));
            review.setSatisfied(rset.getInt("SATISFIED"));
            review.setReviewContent(rset.getString("REVIEW_CONTENT"));
            review.setUserId(rset.getString("USER_ID"));
            review.setReplyDate(rset.getDate("REPLY_DATE"));
            review.setOrderTime(rset.getDate("ORDER_TIME"));

            reviewSearchList.add(review);
```

메뉴리스트 – multipart form

황준호

전송후 서블릿에서 받아주는 형태 확인

```java
protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {

    Map<String,String[]> requestMap = request.getParameterMap();
    Set<String> keySet = requestMap.keySet();
    Iterator<String> keyIter = keySet.iterator();
    System.out.println("keySet을 출력합니다 " + keySet);
//  while(keyIter.hasNext()) {
//      String key = keyIter.next();
//      String[] value = requestMap.get(key);
//
//      System.out.println("key : " + key);
//      for(int i =0; i < value.length; i++) {
//          System.out.println("value[" + i + "] =" + value[i]);
//      }
//  }

    /* 파라미터로 전달된 키 목록만도 확인할 수 있따.
     * -> request.getParameterNames()를 이용
     * Enumeration : 목록화 시킨다. 즉 rset느낌
     */
//  Enumeration<String> names = request.getParameterNames();
//  while(names.hasMoreElements()) {
//      System.out.println(names.nextElement());
//  }
//  String menuName = request.getParameter("menuName");
//  int price = Integer.parseInt(request.getParameter("price"));
//  String salesYN = request.getParameter("salesYN");
//  String menuExplain = request.getParameter("menuExplain");


        System.out.println("파일 저장 ROOT 경로: "+ rootLocation);
        System.out.println("최대 업로드 파일 용량: "+ maxFileSize);
        System.out.println("인코딩 방식: "+ encodingType);

        String fileUploadDirectory = rootLocation + "/semi";

        File directory = new File(fileUploadDirectory);

        //파일 저장경로가 존재하지 않는 경우 디렉토리를 생성하자
        if(!directory.exists()) {
            /* 폴더를 한개만 생성할거면 mkdir, 상위 폴더도 존재하지않으면 한번에 생성하라는 의미로 mkdirs를 이용하자*/
            System.out.println("폴더 생성 : "+ directory.mkdirs());

        }

    /*
     * 최종적으로 request를 parsing 하고 파일을 저장한뒤 필요한 내용을 담을 리스트와 맵이다.
```

```java
    }

    public String getMenuname(MenuListDTO menuList) {

        Connection con = getConnection();
        String result = "";

        result = menuListDAO.getMenuname(con,menuList);

        close(con);

        return result;
    }

    public int updateBoard(MenuListDTO menuList, PictureDTO pictureDTO, StoreInfoDTO storeInfoDTO) {

        Connection con = getConnection();
        int result = 0;

        result = menuListDAO.updateBoard(con,menuList,storeInfoDTO);

        close(con);

        return result;
    }

public int deleteMenu(String MENUCode, StoreInfoDTO storeInfoDTO) {

        Connection con = getConnection();

        int result = menuListDAO.deleteMenu(MENUCode,storeInfoDTO,con);

        close(con);

        return result;
    }
```

```java
@WebServlet("/menu/delete")
public class DeleteMenuServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {

        HttpSession session = request.getSession();
        OwnerDTO ownerDTO = (OwnerDTO) session.getAttribute("ownerDTO");
        LicenseManagerDTO lmDTO = (LicenseManagerDTO) session.getAttribute("lmDTO");
        StoreInfoDTO storeInfoDTO = (StoreInfoDTO) session.getAttribute("storeInfoDTO");

        String MENUCode = request.getParameter("MENUCode");

        String path= "";
        System.out.println("boardCode in servlet: "+MENUCode);

        int deletecheck = new MenuListService().deleteMenu(MENUCode,storeInfoDTO);

        if(deletecheck >0) {
            path="/WEB-INF/views/owner/MenuManage.jsp";
        }else {
            request.setAttribute("message", "문의 상세글을 불러오는데 실패하셨습니다. 다시 페이지를 시작해주세요!");
            path="/WEB-INF/views/common/faild.jsp";
        }

        request.getRequestDispatcher(path).forward(request, response);
```